

Computer Architecture

CPU Design&Simulation Project Guide

Designed By: Ayyub Murjan

Date: 30-01-2026

1. System Architecture: Datapath and Memory Management

This CPU utilizes a structured organization that dictates data movement and memory allocation.

1. **Shared Internal Bus:** All primary components are interfaced via a single 8-bit bidirectional bus. To prevent bus contention, every component uses high-impedance **tri-state buffers**. These buffers ensure that only one source drives the bus at any given time, while multiple components may act as receivers.
2. **Memory Segmentation (The 0xF Segment):** The system employs memory mapping to separate logic types within the 256-byte RAM.
 - **Instruction Segment:** Execution begins at address 0x00.
 - **Data Segment:** Operands and variables are mapped to the bottom 16 bytes of memory (starting at 0xF0). A hardware-level **Constant F Masker** applies an offset to 4-bit operands, ensuring a physical separation between executable code and data variables.
3. **Sequential Synchronization:** To ensure data integrity and satisfy register setup times, the system follows a multiphase timing protocol. Valid data is placed on the bus during one clock cycle and is latched by the destination register on the subsequent cycle, preventing race conditions.

2. Functional Structure: Core Hardware Components

The hardware is categorized into three functional groups based on their role in the instruction cycle.

1. **Arithmetic Logic Unit (ALU):** The ALU is the combinational logic core of the CPU, capable of executing 14 distinct arithmetic and bitwise operations. It generates four status flags based on the result:
 - **Zero (Z):** Asserted if the output is 0x00.
 - **Negative (N):** Asserted if the Most Significant Bit (MSB) of the result is 1.
 - **Carry (C) & Overflow (V):** Asserted during arithmetic operations where the result exceeds the bit-width capacity or violates signed-integer logic.
2. **Register File:** This group manages internal data storage and operand sourcing.
 - **Accumulator (Reg A):** The primary register in this architecture. It serves as the mandatory source or destination for all memory-to-register transfers and ALU operations.
 - **General Purpose Registers (Reg B, C, D):** These registers provide internal storage for intermediate operands, reducing the frequency of memory access cycles.
3. **Control and Navigation Registers:**
 - **Program Counter (PC):** An 8-bit register that stores the address of the next instruction to be fetched.
 - **Instruction Register (IR):** Latches the 8-bit opcode and operand fetched from memory, routing the opcode bits to the control logic and the operand bits to the address bus.

- **MAR & MBR:** The Memory Address Register and Memory Buffer Register interface with the RAM to stabilize signals during read and write operations.

3. Control Unit: Microprogrammed Logic

The Control Unit is the finite state machine that automates the hardware through sequenced micro-operations.

1. **Microprogrammed ROM:** The system is driven by a Control ROM acting as an instruction decoder. It stores 24-bit **Control Words**. When an opcode is latched into the IR, the ROM outputs a specific bit-pattern that actuates the load-enables and tri-state buffers of the datapath.
2. **Timing and State Sequencing:** Operations are synchronized by a **3-bit Step Counter**, allowing for 8 distinct T-states.
 - **Fetch Phase:** T0 through T2 are universal across all instructions, dedicated to retrieving the opcode from RAM and incrementing the PC.
 - **Execution Phase:** T3 through T6 execute the specific logic of the opcode.
3. **Run-State Logic:**
 - **State Initialization:** A **D Flip-Flop** circuit manages the run state. It latches a high signal to the counter's enable pin upon a manual START pulse.
 - **Hardware Halt:** Upon detecting the **HALT** opcode, the Control ROM asserts a signal to the Flip-Flop's clear pin, disabling the Step Counter and terminating execution.

4. System Initialization: Loading the Microcode

The CPU requires its internal logic definitions to be loaded into the Control ROM before operation.

1. **The ROM Image:** A predefined .rom file is provided with this documentation. It contains the microcode definitions for all 16 opcodes.
2. **Installation:** In Logisim, right-click the **ROM component** and select "**Load Image...**" Select the provided file to populate the Control Word table.
3. **Verification:** Confirm that hex values (e.g., 200021) appear in the ROM grid.

5. Operating Protocol: The Launch Sequence

To execute a program, the following hardware initialization sequence must be followed:

1. **Clock Configuration:** Under the **Simulate** menu, set the **Tick Frequency** (4 Hz for observation, 1024 Hz for performance).
2. **System Reset:** Click the manual **Reset** button. This clears the PC to 0x00, the Step Counter to 0, and initializes the registers.
3. **Run Trigger:** Click the **START** button to enable the D Flip-Flop.
4. **Automation:** Select **Simulate** → **Ticks Enabled**. The CPU will begin the fetch-execute cycle automatically until a termination state is reached.

6. Programming Interface: Opcode Dictionary

Enter instructions into the **RAM** by first-digit (Opcode) and second-digit (Operand/Offset).

Opcode	Mnemonic	Functional Description
0x	ADD	Adds Reg B to Reg A; result stored in Reg A.
1x	SUB	Subtracts Reg B from Reg A; result stored in Reg A.
2x	AND	Bitwise AND between Reg A and Reg B.
3x	OR	Bitwise OR between Reg A and Reg B.
4x	NOT	Bitwise inversion of Reg A.
5x	SHL	Logical Shift Left of Reg A (Arithmetic multiplication).
6x	SHR	Logical Shift Right of Reg A (Arithmetic division).
7x	INC	Increment Reg A by 1.
8x	DEC	Decrement Reg A by 1.
9x	MOV B, A	Transfer contents of Reg A to Reg B.
Ax	LOAD A	Move data from RAM address 0xFx to Reg A.
Bx	JMP	Set PC to address 0x0x.
Cx	STORE A	Move contents of Reg A to RAM address 0xFx.
Dx	MOV C, A	Transfer contents of Reg A to Reg C.
Ex	MOV D, A	Transfer contents of Reg A to Reg D.
Fx	HALT	Assert Halt signal to terminate execution.

The second digit “x” points to the 16 memory addresses: F0-FF(using a 0-F value).

7. Performance Validation: Use Cases

Enter these hex values into RAM starting at address 0x00 to verify system logic.

Use Case A: Signed Arithmetic and Flag Generation

- **Goal:** Execute
$$5-12=-75-12=-7$$
- **RAM Instructions:** A1, 90, A0, 11, C2, F0
- **Data Inputs:** Address **F0** = 05, Address **F1** = 0C (12).
- **Expected Result:** RAM address **F2** updates to **F9**. The **Negative Flag** is asserted.

Use Case B: Register File Addressing

- **Goal:** Verify independent write-enables for C and D.
- **RAM Instructions:** A0, 90, A1, D0, A2, E0, F0
- **Data Inputs:** Address **F0** = 11, **F1** = 22, **F2** = 33.
- **Expected Result:** Registers **B, C, and D** store 11, 22, and 33 respectively.

Use Case C: Unary Math and Termination

- **Goal:** Calculate
$$(3 \times 2) + 1 = 7$$
- **RAM Instructions:** A0, 50, 70, C1, F0
- **Data Inputs:** Address **F0** = 03.
- **Expected Result:** RAM address **F1** updates to **07**. The system halts.

Use Case D: Bitwise Logical Isolation (Masking)

- **Goal:** Validate the transparency of the bitwise AND operation by isolating the lower nibble (4 bits) of a data byte.
- **Logic:** 1010 1010 (0xAA) AND 0000 1111 (0x0F) should result in 0000 1010 (0x0A).
- **RAM Instructions:** A0, 90, A1, 21, C2, F0
- **Data Inputs:** Address **F0** = AA, Address **F1** = 0F.
- **Outcome:** RAM address **F2** updates to **0A**. This proves the correct integration of the logical AND gate within the ALU and the stability of the dual-register sourcing.

Use Case E: Address Flow Control (Unconditional Branching)

- **Goal:** Verify the Program Counter's (PC) ability to override sequential increments by ingesting an immediate address from the Instruction Register.
- **Logic:** Skip an INC instruction to prove that the execution flow has been redirected.
- **RAM Instructions:**
 - 00: A0 (Load 05)
 - 01: B3 (JMP to Address 03)
 - 02: 70 (Increment A — **Should be skipped**)
 - 03: C1 (Store result)
 - 04: F0 (Halt)
- **Data Inputs:** Address **F0** = 05.

- **Outcome:** RAM address **F1** will result in **05** rather than 06. This confirms the **Ld_PC** control signal correctly actuates the PC's parallel load function.

Use Case F: Recursive Variable Processing (Cumulative Addition)

- **Goal:** Demonstrate the robustness of the bidirectional datapath by retrieving a value, processing it, and updating the same memory address.
- **Logic:** Implement a "Running Total" by adding a constant to a memory variable.
- **RAM Instructions:** A0, 91, A1, 01, C0, F0
- **Data Inputs:** Address **F0** (Variable) = 0A (10), Address **F1** (Constant) = 05.
- **Outcome:** RAM address **F0** updates from 0A to **0F**. This confirms that the memory mapping logic handles simultaneous address bus and data bus activity across multiple cycles without data corruption.

Use Case G: Extreme Arithmetic (Carry and Rollover)

- **Goal:** Test the ALU's limit-handling by forcing an arithmetic rollover.
- **Logic:** $0xFF + 0x01 = 0x00$.
- **RAM Instructions:** A0, 70, C1, F0
- **Data Inputs:** Address **F0** = FF.
- **Outcome:** RAM address **F1** updates to **00**. Both the **Zero Flag (Z)** and **Carry Flag (C)** are asserted. This validates the status flag combinational logic and the precision of the 8-bit wrap-around.

NOTE: Status Flags are active for the state/Instruction that triggers the flag. This can be observed by manually ticking or selecting a slower tick rate.